

김효진 협업일지 요약

중급프로젝트 RFP RAG · 기록 기간 2026-07-14 ~ 2026-07-31

역할 요약

김효진님은 프로젝트의 서비스·앱 통합 및 품질 검증 담당자로서 초기 개발환경과 기준선 코드를 정리하고, RFP 업무 흐름을 사용자 시나리오와 UI로 구체화했다. 이후 Naive·Advanced RAG를 실제로 실행해 Retrieval과 Generation을 W&B에서 관측하고, Golden Set 기반 평가와 RAGAS·LLM Judge를 연결했다. 후반에는 Advanced RAG를 UI·백엔드에 통합하고 문서 업로드·인덱싱, Q&A, 근거 확인, 업무 카드와 평가 화면을 하나의 사용 흐름으로 구성했으며, Playwright·pytest·프론트엔드 빌드로 통합 상태를 검증했다.

핵심 기여

1. 프로젝트 초기 환경과 기준선 정리

- Kick-off 이후 GCP와 Git 연결, 베이스라인 working version 확보를 초기 목표로 설정했다.
- 팀의 EDA 진행 상황과 업무 범위를 공유하고, src를 한곳에서 시작한 뒤 복잡도가 증가하면 기능별로 나누는 디렉터리 구조를 협의했다.
- 팀원 변동에 맞춰 업무 분장을 다시 조정하고, 검색·임베딩과 Generation 영역의 담당 범위를 정리했다.
- 로컬 베이스라인 코드를 Git 저장소 구조에 맞게 재구성하고 실행 여부를 확인했다.
- 브랜치를 main, dev, 기능·실험·버그 수정 브랜치로 구분해 안정본과 개발 작업을 분리하는 방향을 제안했다.
- GCP 접속이 PC 환경 문제로 원활하지 않았던 경험을 기록하고, 필요할 경우 Mac 환경을 이용하는 대안을 검토했다.

2. RFP 업무 흐름과 사용자 시나리오 설계

RFP 서비스를 단순 질의응답 화면으로 보지 않고, 실제 제안 업무 흐름을 기준으로 앱 구조를 설계했다.

업무 단계	사용자 관점의 의미
공고	참여할 사업과 RFP 확인
접수	계획서·제안서와 제출서류 준비
평가	선정평가 기준과 위험 요소 검토
협약	발주·계약 조건 확인
수행	착수 후 산출물과 이행 항목 관리

- 핵심 사용자를 각 회사의 과제 제안 담당자로 정의했다.

- 서비스 전달 방식(Web/App), 인프라(Cloud/Local), MVP 범위를 검토했다.
- 기존 앱을 기반으로 초기 UI를 만들고 사용자의 작업·사고 순서에 맞춰 다음 탭을 구성했다.

탭	목적
Overview	사업과 문서의 핵심 정보 요약
Go/No-Go	입찰 참여 판단과 위험 확인
실행준비	제출서류·일정·준비 항목 관리
요구사항	기능·기술·계약 요구사항 탐색
AI 질문	RAG 기반 질의응답과 근거 확인

이 설계는 UX/UI가 화면 장식이 아니라 사용자의 실제 업무 환경과 판단 순서를 이해하는 데서 시작해야 한다는 기준에 기반했다.

3. Naive RAG 실행과 W&B 관측 환경 구축

- 현재 환경에서 `main.py`를 직접 실행해 Naive RAG의 동작을 확인했다.
- 난이도별 RAG 테스트 문서를 3개씩 선정하고, 총 9개 PDF를 로컬 Chroma에 인덱싱했다.
- Retrieval과 Generation을 W&B에서 분리 관측할 수 있도록 코드를 추가했다.
- W&B 계정·API key 설정과 코드 적용 절차를 정리하고 `W&B_GUIDE` 문서를 작성했다.
- 테스트와 실행 결과를 추적할 수 있도록 Weave Trace와 pytest 결과 자동 업로드 방향을 정리했다.
- Mac에서 PC Ubuntu 환경으로 접속하기 위한 Tailscale-SSH 설정을 실험하고 두 개의 사용 가이드를 작성했다.

4. Golden Set 파일럿과 초기 문제 발견

- 9개 문서를 대상으로 Golden Set 102문항을 생성해 1차 파이프라인 테스트를 수행했다.
- 첫 모델 실행에서 9개 오류를 발견했다. 오류 유형은 일부만 답변함, 답변 없음, 유사어·복합 문장 이해 실패였다.
- 답변이 지나치게 장황하고 질문의 초점에서 벗어나는 문제도 확인했다.
- 단순히 모델 응답을 보는 것에서 그치지 않고 Retrieval과 Generation을 분리해 어느 단계에서 문제가 발생했는지 관측하도록 했다.
- 파일럿 결과를 바탕으로 Golden Set, CI, UI 테스트 관련 가이드와 작업 문서를 정리했다.

5. UI 기능·시인성·근거 확인 개선

- 탭별 기능을 점검하고 폰트 크기와 레이아웃 균형을 조정해 시인성을 개선했다.
- Overview와 업무 탭이 서로 다른 위험·제출물 개수를 보여주던 문제를 해결하기 위해 데이터 기준을 통합했다.
- Overview에 탭별 요약을 추가하고 목차와 파일 목록을 접이식으로 바꿔 정보 밀도를 조절했다.
- 위험 분석 범위를 위약금과 계약상 불이익까지 넓혔다.
- 분석 전 상태가 0건으로 오해되지 않도록 위험 상태를 **미확인**, **위험 있음**, **문제없음**으로 구분했다.
- 제출물 검색어를 여러 표현으로 확장하고 빈 결과가 캐시에 남지 않게 해 누락 가능성을 줄였다.
- 입찰 단계의 제출서류와 계약 이후 사업 산출물을 분리해 사용자가 준비 시점을 구분할 수 있게 했다.
- 파일 목록의 검색·정렬 기능을 정리하고 Evidence 패널은 폭 저장과 원문 중심 표시를 적용해 근거 확인성을 높였다.

6. Playwright·CI 기반 UI 회귀 검증

- UI 수정 후 수동 확인만 하지 않고 Playwright 회귀 테스트와 스냅샷을 추가했다.
- 테스트를 GitHub Actions CI에 연결해 변경 시 핵심 화면이 깨지는지 자동으로 확인하도록 했다.

- 백엔드 smoke test, pytest, Playwright UI 회귀 테스트와 프론트엔드 빌드를 함께 실행해 통합 상태를 검증했다.
- 평가 API, RAG 클라이언트, 백엔드 모델 변경에 대한 회귀 테스트를 정비했다.

7. Advanced RAG 통합과 평가 파이프라인

- Naive와 Advanced 두 DB를 분석하고, 메인 RAG 파이프라인은 최대한 보존하면서 UI 디렉터리가 실행환경·평가 harness·API·UI shell 역할을 하도록 구조를 설계했다.
- Advanced 코드의 import와 파일 의존성을 조사하고 [src/main_rag/](#) 구조를 만든 뒤 필요한 코드를 포팅했다.
- 의존성과 절대경로 설정을 정리하고 기존 Advanced 테스트를 이식했다.
- Advanced 전처리·칭킹·Dense 인덱스와 Retriever를 연결했다.
- BidMateRAGSession, 단일 질문 CLI smoke test, Retrieval Adapter, Q&A Adapter를 구성했다.
- Backend의 `RAGClient.answer()`와 Conversation session을 연결하고, Workspace 카드 검색기와 Q&A UI를 교체했다.
- 문서 업로드·인덱싱과 실제 백엔드 UI smoke test를 보완해 사용자 입력부터 답변·근거 표시까지의 흐름을 확인했다.

8. 104문항 평가와 모델 비교

- Golden 질문 104개를 전부 생성해 오류 없이 평가 입력을 만들었다.
- Answerable 83개, Unanswerable 21개로 구분하고 잘못된 citation 0개를 확인했다.
- 답변 생성은 `gpt-5-nano`, RAGAS와 LLM Judge는 호환성과 자기평가 편향을 고려해 `gpt-4o-mini`로 분리했다.
- Top-10 검색 결과로 답변을 생성하고 RAGAS 상세 결과와 평가 보고서를 작성했다.

평가 항목	결과
Golden 질문 생성	104/104 성공, 오류 0
Answerable / Unanswerable	83 / 21
잘못된 citation	0개
Answerability accuracy	0.7885
Required fact coverage	0.6079
Numeric recall	0.7003
Numeric precision	0.5893
RAGAS faithfulness	0.8142
RAGAS answer relevancy	0.3364
LLM Judge faithfulness	0.8894
LLM Judge answer relevancy	0.9091
회귀 테스트	47개 통과

- 품질·완전성은 `gpt-5-nano`, 속도·간결성은 일지 기록 기준 Gemini Flash 계열이 강점이라고 비교했다.
- Flash-Lite는 빠르지만 숫자 precision이 불안정하다는 점을 확인했다.
- 평가가 보수적으로 보여도 결과를 숨기지 않고 답변 가능성 판단 로직과 평가 UI 표시를 보완했다.
- 병렬 실행, 중단 후 재개, API 비용과 rate limit 관리 원칙을 실행 정책으로 문서화했다.

9. 범위 조정과 최종 산출물

- 최신 **main**을 다시 받아 UI를 추가 개선하려 했으나, 남은 작업량이 최소 하루 이상이라고 판단했다.
- 강사에게 범위를 확인한 뒤 마감 직전의 큰 변경은 중단하고 안정적인 현재 결과를 유지했다.
- 최종 개발보고서의 리서치·앱 UI 파트를 작성하고 자유 형식 보고서와 발표 자료를 만들었다.

기간별 작업 흐름

기간	주요 작업	협업 결과
7/14~7/16	GCP·Git·기준선 코드·업무 분장·브랜치 구조	팀 개발환경과 기준선 정리
7/20~7/21	RFP 업무 흐름·사용자 시나리오·앱 탭 설계	RAG Copilot MVP 구조 확정
7/22~7/24	Naive 실행, 9개 PDF 인덱싱, W&B·Tailscale 문서	파이프라인 관측 및 원격 개발 절차 마련
7/23~7/27	Golden Set 파일럿, UI·위험·제출물·근거 패널 개선	초기 오류 유형 발견과 사용성 보완
7/27~7/29	Advanced RAG 포팅·평가·백엔드/UI 통합	104문항 평가와 통합 smoke test 완료
7/30~7/31	변경 범위 조정, 보고서·PPT	안정본 유지와 최종 산출물 작성

문제 해결에서 드러난 강점

- 1 업무 흐름을 UI 구조로 번역했다. RFP 공고부터 수행까지 실제 사용자의 판단 순서를 탭과 기능으로 구체화했다.
- 2 관측 가능한 파이프라인을 만들었다. Retrieval과 Generation을 분리해 W&B·Weave에서 보고, 문제가 어느 구간에서 생겼는지 확인할 수 있게 했다.
- 3 평가를 서비스 화면과 연결했다. Golden Set·RAGAS·LLM Judge 결과를 별도 실험으로 끝내지 않고 평가 UI와 답변 가능성 표시로 연결했다.
- 4 자동 회귀 검증을 적용했다. Playwright, pytest, CI, 프론트엔드 빌드를 사용해 UI와 백엔드 변경을 함께 확인했다.
- 5 마감 직전 범위를 통제했다. 추가 개선의 작업량을 확인하고 강사와 협의해 안정본을 유지하는 결정을 내렸다.